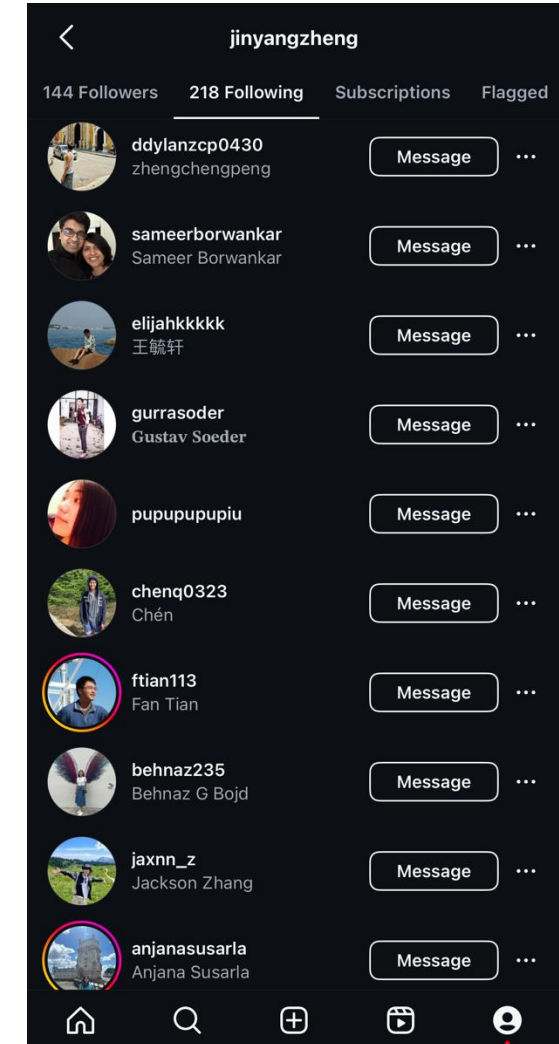
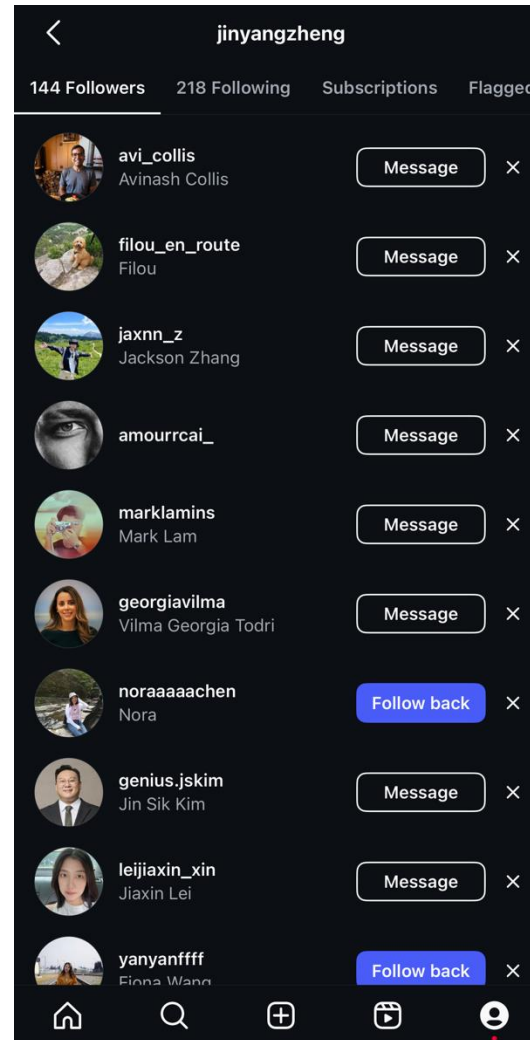
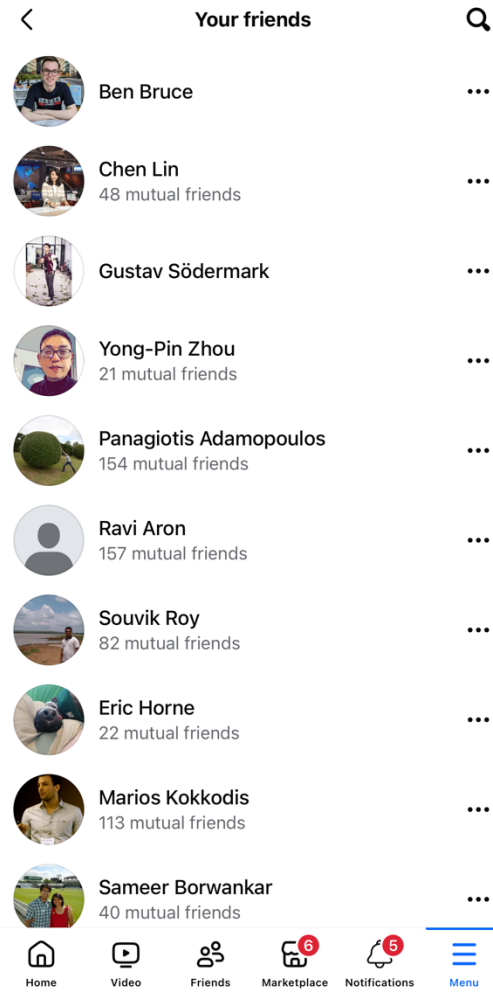


S10: Social Network Basic and Representation

Social media network



Combine everyone's (fb) friend list

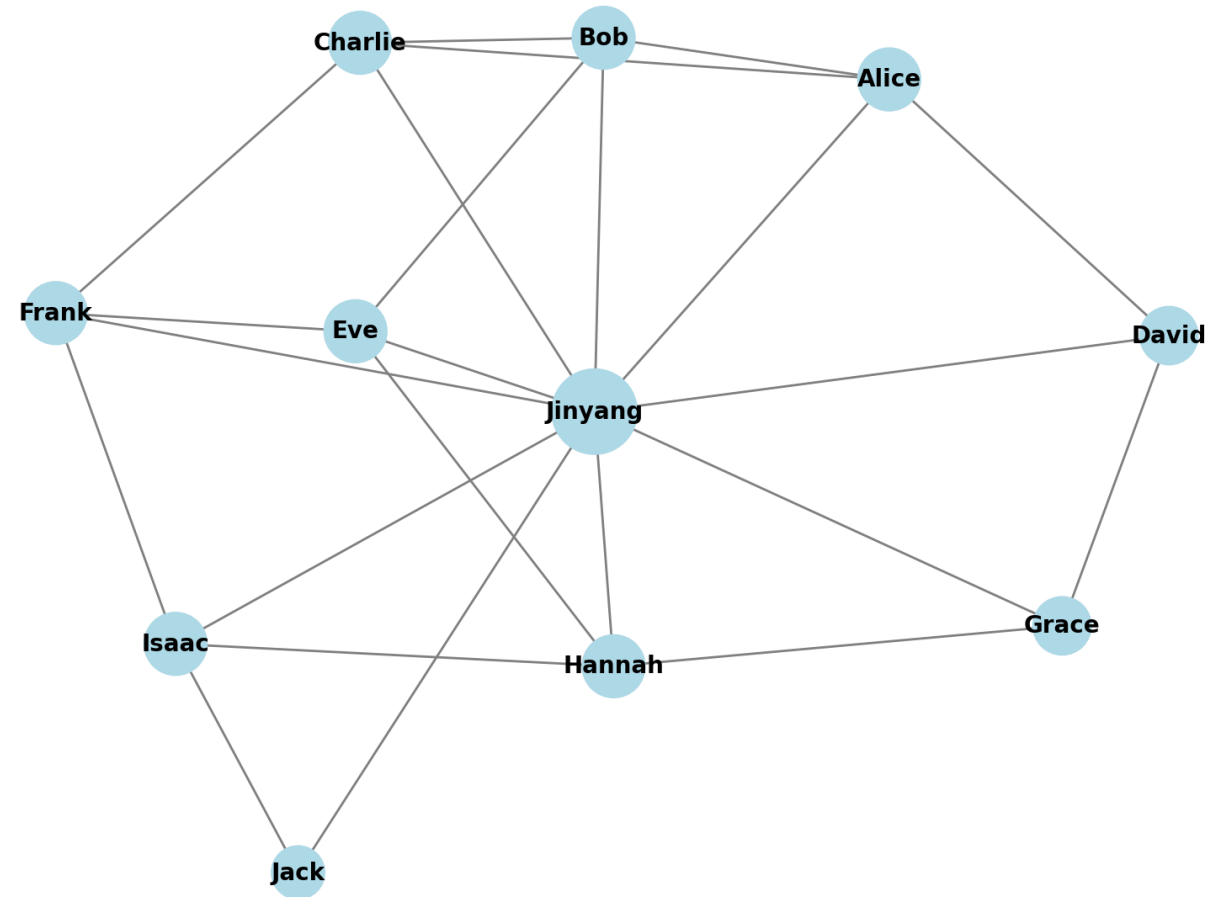
☐	Person	Friends
1	Jinyang	Alice, Bob, Charlie, David, Eve, Frank, Grace, Hannah, Isaac, Jack
2	Alice	Bob, Charlie, David, Jinyang
3	Bob	Alice, Charlie, Eve, Jinyang
4	Charlie	Alice, Bob, Frank, Jinyang
5	David	Alice, Grace, Jinyang
6	Eve	Bob, Frank, Hannah, Jinyang
7	Frank	Charlie, Eve, Isaac, Jinyang
8	Grace	David, Hannah, Jinyang
9	Hannah	Eve, Grace, Isaac, Jinyang
10	Isaac	Frank, Hannah, Jack, Jinyang
11	Jack	Isaac, Jinyang

- **Person**
- **Friendship**

Combine everyone's (fb) friend list

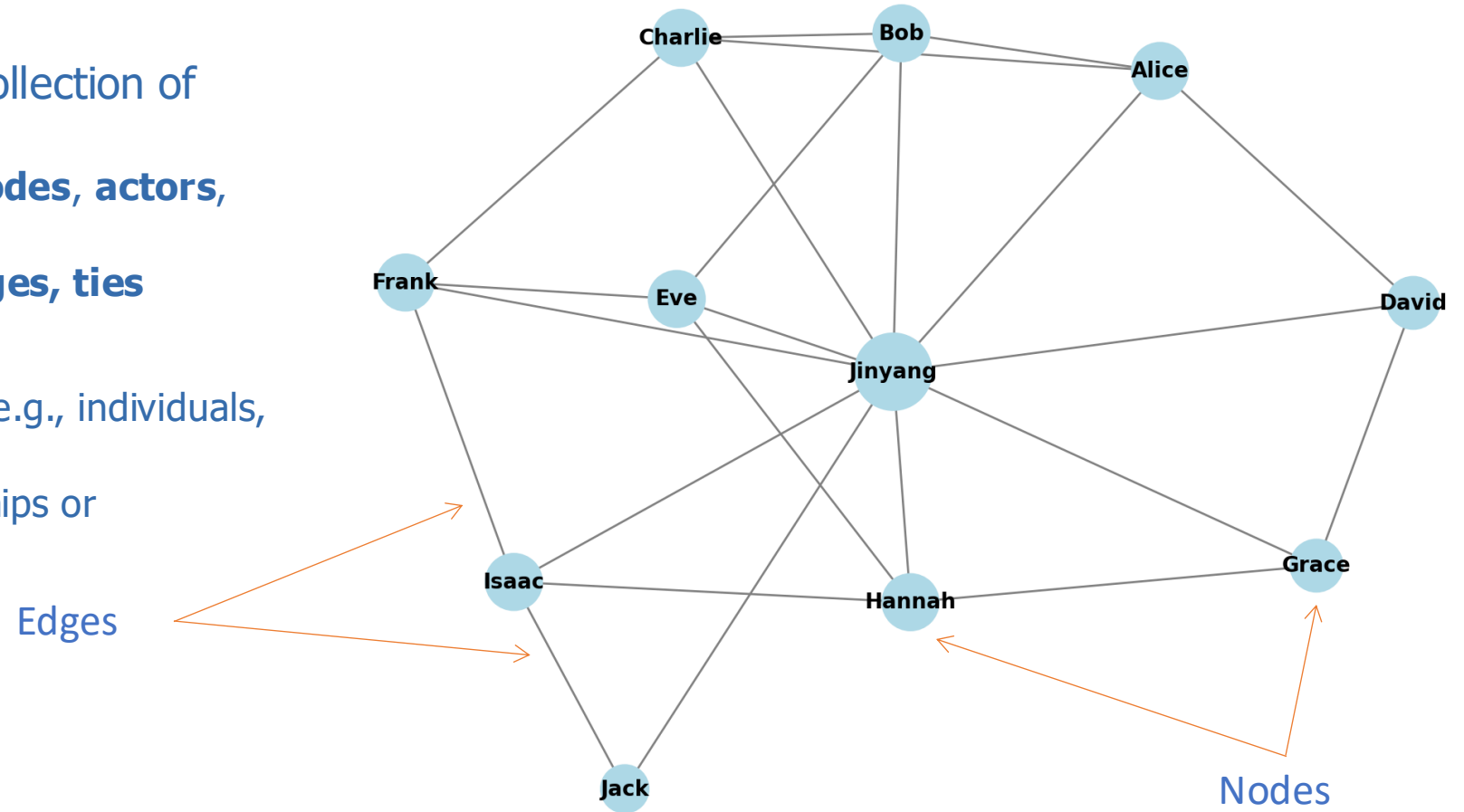
☐	Person	Friends
1	Jinyang	Alice, Bob, Charlie, David, Eve, Frank, Grace, Hannah, Isaac, Jack
2	Alice	Bob, Charlie, David, Jinyang
3	Bob	Alice, Charlie, Eve, Jinyang
4	Charlie	Alice, Bob, Frank, Jinyang
5	David	Alice, Grace, Jinyang
6	Eve	Bob, Frank, Hannah, Jinyang
7	Frank	Charlie, Eve, Isaac, Jinyang
8	Grace	David, Hannah, Jinyang
9	Hannah	Eve, Grace, Isaac, Jinyang
10	Isaac	Frank, Hannah, Jack, Jinyang
11	Jack	Isaac, Jinyang

- **Person** $\leftrightarrow$ **point**
- **Friendship** $\leftrightarrow$ **line**



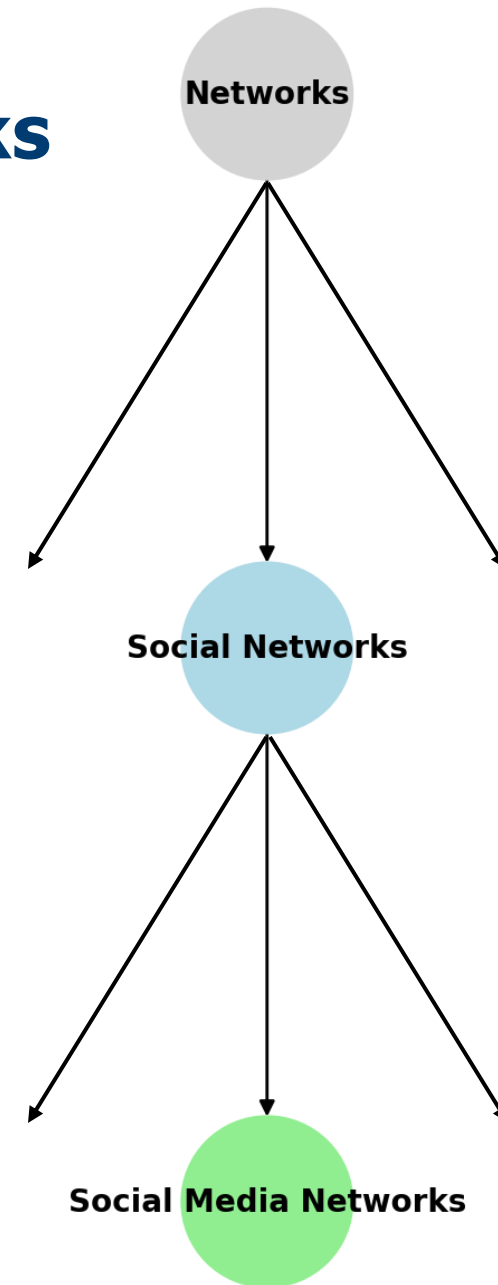
Social network

- A **network** is a graph, or a collection of **points** connected by **lines**
 - **Points** are referred to as **nodes**, **actors**, **vertices** (plural of **vertex**)
 - **Lines** are referred to as **edges**, **ties**
- **Social network**
 - **Nodes:** Represent entities (e.g., individuals, organizations).
 - **Edges:** Represent relationships or interactions between nodes.



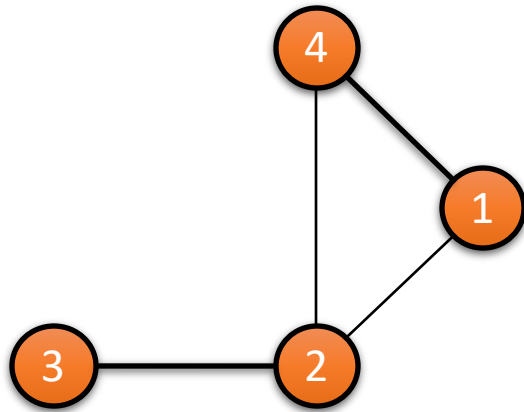
Social networks vs. social media networks

- **Nodes** could be individuals and/or collectivities
- **Individuals**
 - persons
 - other animals
- **Collectivities**
 - organizations, departments, teams, troops
 - countries, cities
 - species
- **Edges** indicate any types of internode relations
- **Kinship**, e.g., mother of
- **Other social role-based**, e.g., boss of, friend of
- **Cognitive/perceptual**, e.g., knows, aware of
- **Affective**, e.g., likes, trusts
- **Interactions**, e.g., give advice, talk to
- **Affiliations**, e.g., belong to same clubs, is physically near

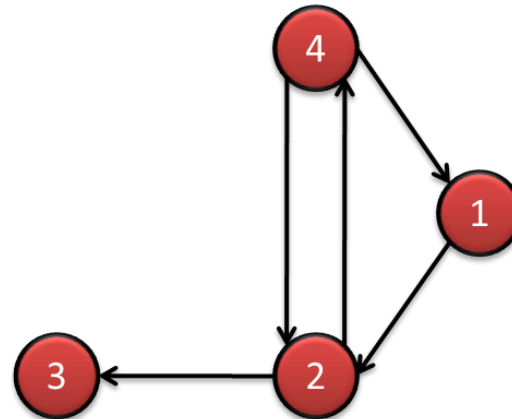


Directedness of edges

- Undirected edges
- E.g., Friendship



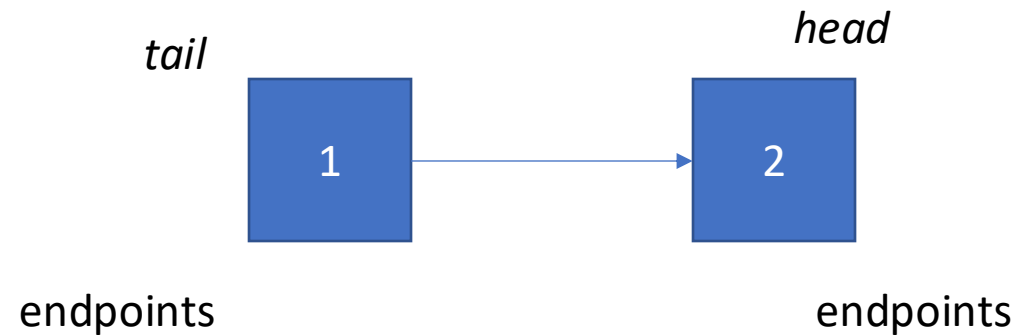
- Directed edges
- E.g., follower-followee relationships.



- Which type of edges captures facebook's friendship?
- What about Instagram?
- What does it imply in information transmission?

Directed edge

- There is an edge from 1 to 2, but not from 2 to 1.
- Vertex 1 is called *tail*, and vertex 2 is called *head*.



- How many directed edges to the most can you have between a pair of nodes?
- What about undirected edges?

MGP4b

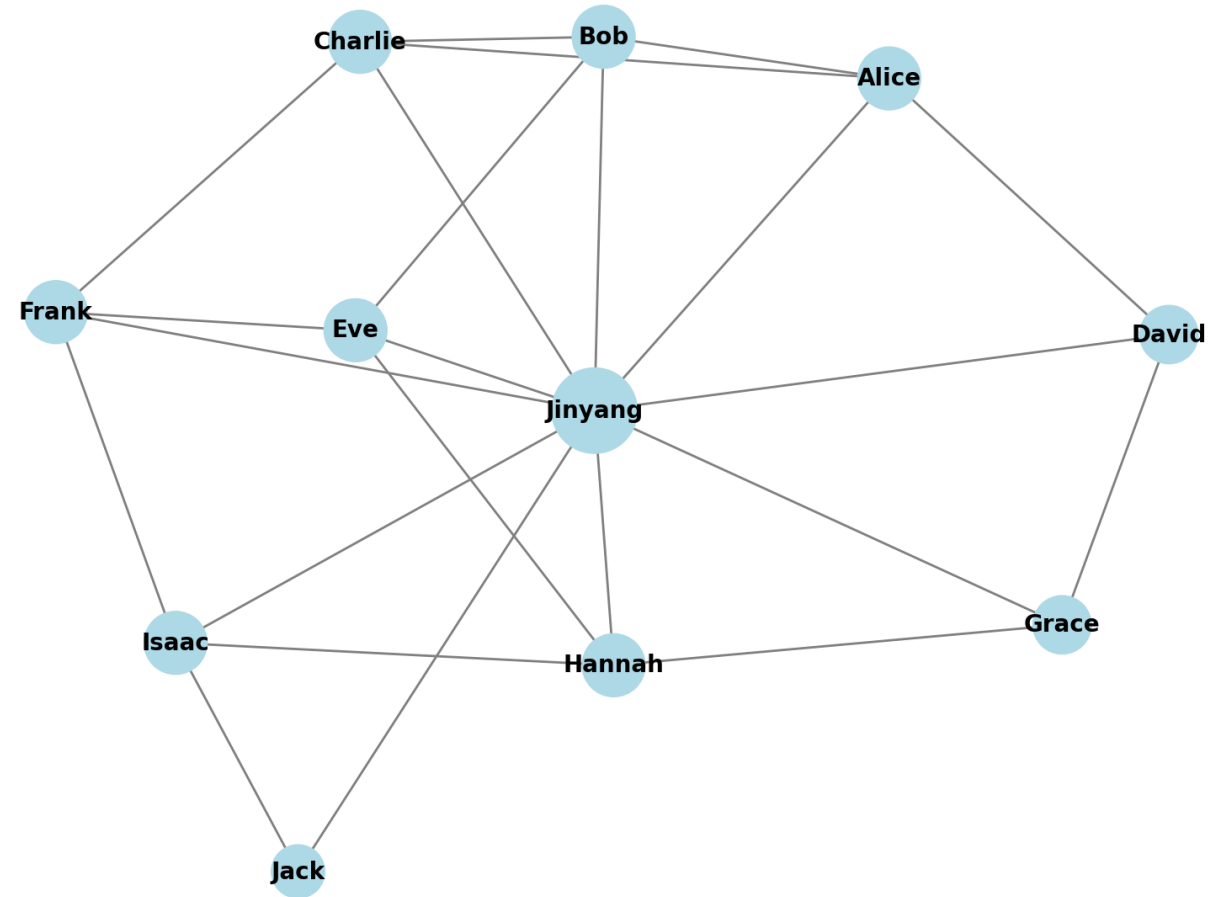
- On the back of the group no. paper, please draw:
- A social network that represent the “know” social relations of your group before you attended our course
- Please consider to use directed or undirected edges

Social networks analyses (SNA)

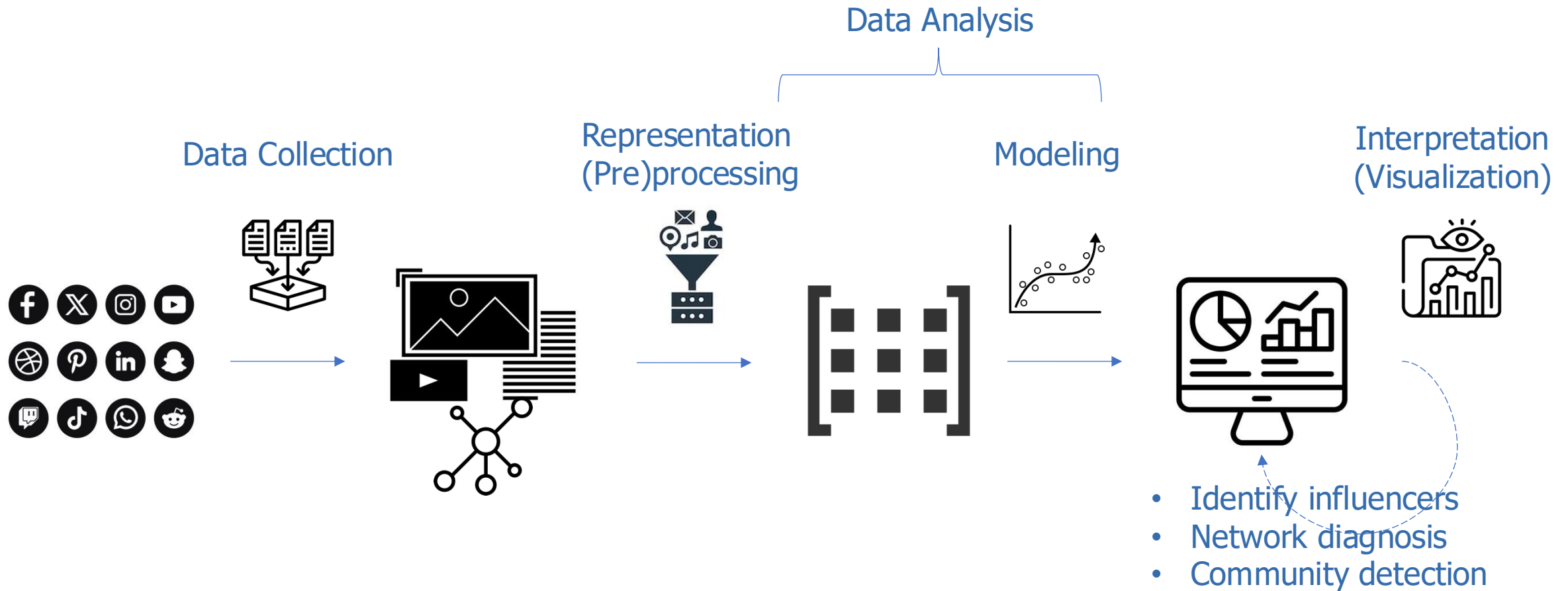
- **Network diagnosis:** Understand the structure of relationship among entities
- **Identify influential nodes (influencers)**
- **Detect communities**
- Practical Uses:
 - Seeding strategy, influencer strategy
 - Marketing and customer segmentation
 - Combating misinformation
 - Disease spread modeling
- Applications: business, sociology, epidemiology, etc.

How to analyze social networks?

- Graph representation is straightforward and intuitive
- but it is **unstructured** – cannot be effectively manipulated using mathematical and computational tools



Analyze unstructured data



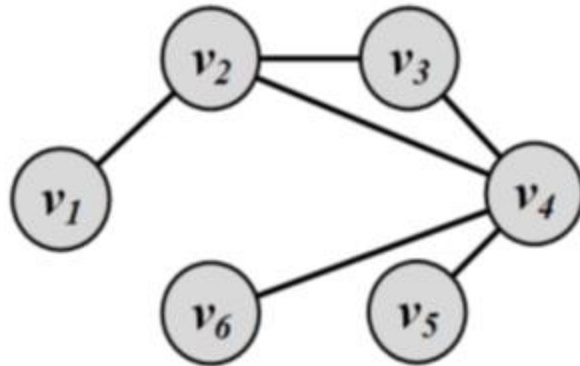
Representation

- We are seeking representations that can store these two sets in a way such that
 - Does not lose information
 - Can be manipulated easily by computers
 - Can have mathematical methods applied easily
- Adjacency List
- Adjacency Matrix (Sociomatrix)
- Edge List



Adjacency list for undirected networks

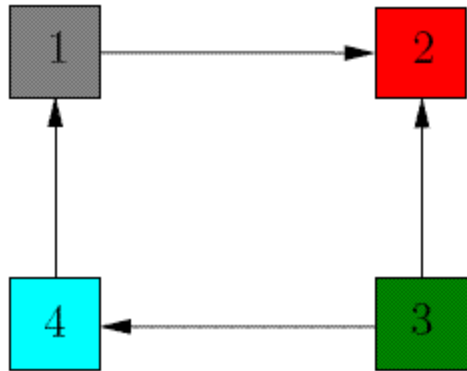
- In an adjacency list for every node, we maintain a list of all the nodes that it is connected to
- The list is usually sorted based on the node order or other preferences



Node	Connected To
v_1	v_2
v_2	v_1, v_3, v_4
v_3	v_2, v_4
v_4	v_2, v_3, v_5, v_6
v_5	v_4
v_6	v_4

Adjacency list for directed networks

- In an adjacency list for every node, we maintain a list of all the nodes that it head to
- Can you compute on this restructured data?

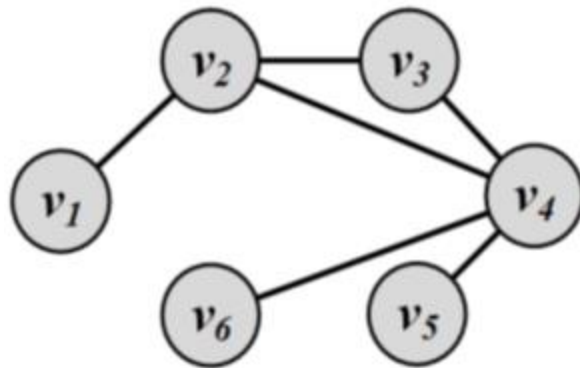


Node	Head to
1	2
2	
3	2,4
4	1

Adjacency matrix (sociomatrix) for undirected networks

- 0/1 indicates the existence of an edge

$$A_{ij} = \begin{cases} 1, & \text{if there is an edge between nodes } v_i \text{ and } v_j \\ 0, & \text{otherwise} \end{cases}$$



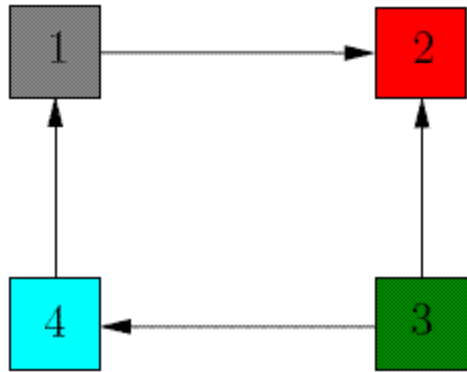
	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	1	0	0	0	0
v_2	1	0	1	1	0	0
v_3	0	1	0	1	0	0
v_4	0	1	1	0	1	1
v_5	0	0	0	1	0	0
v_6	0	0	0	1	0	0

- Diagonal entries are self-links or loops
- Symmetric and squared matrix format

Adjacency matrix (sociomatrix) for directed networks

- 0/1 indicates the existence of an arrow (arrowed edge)

$$A_{ij} = \begin{cases} 1, & \text{if there is an arrow from nodes } v_i \text{ to } v_j \\ 0, & \text{otherwise} \end{cases}$$



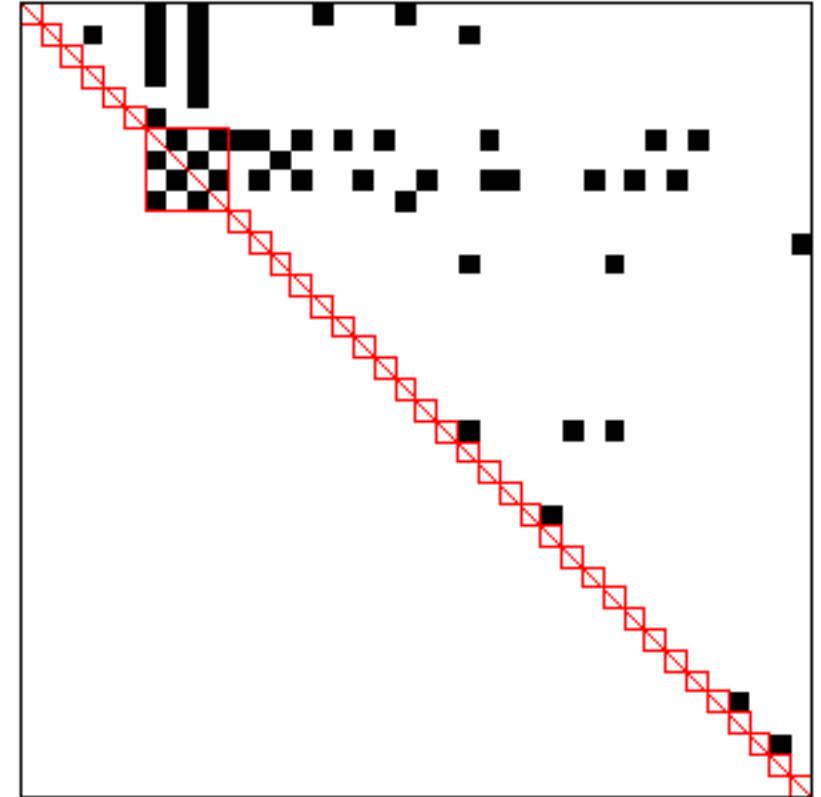
Node	Head to
1	2
2	
3	2,4
4	1

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

- Squared matrix?
- Symmetric?

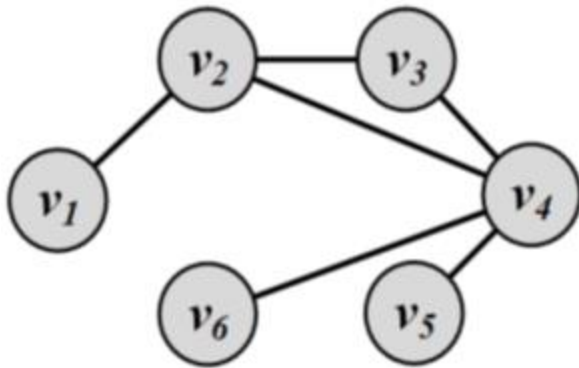
Issues with sociomatrix

- When you have 1million users, what is the size of your sociomatrix?
- What would the matrix look like? How many zero would there be?
- Can we just preserve the info of "1" and suppress those "0"?



Edge list for undirected networks

- In this representation, each element is an edge and is usually represented as (u, v) , denoting that node u is connected to node v via an edge



	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	1	0	0	0	0
v_2	1	0	1	1	0	0
v_3	0	1	0	1	0	0
v_4	0	1	1	0	1	1
v_5	0	0	0	1	0	0
v_6	0	0	0	1	0	0

(v_1, v_2)

(v_2, v_3)

(v_2, v_4)

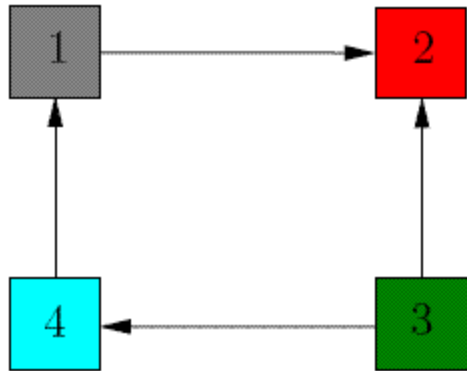
(v_3, v_4)

(v_4, v_5)

(v_4, v_6)

Edge list for directed networks

- In this representation, each element is an arrow (arrowed edge) and is usually represented as (u, v) , denoting that node u head to node v via an arrow
- Can you compute on this representation?
- What if there are unconnected nodes?



Node	Head to
1	2
2	
3	2,4
4	1

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{matrix} (1,2) \\ (4,1) \\ (3,4) \\ (3,2) \end{matrix}$$

Python: Create a network with edge or adj list

- `import networkx as nx`
`import pandas as pd`
- `## edge list`
`edges = [`
 `("Alice", "Bob"),`
 `("Alice", "Charlie"),`
 `("Bob", "David"),`
 `("Charlie", "Eve"),`
 `("David", "Eve")`
`]`
`G = nx.Graph(edges)`
- `DG = nx.DiGraph(edges)`

- `#adj list`
`adj_list = {`
 `"Alice": ["Bob", "Charlie"],`
 `"Bob": ["Alice", "David"],`
 `"Charlie": ["Alice", "Eve"],`
 `"David": ["Bob", "Eve"],`
 `"Eve": ["Charlie", "David"]`
`}`
`G = nx.Graph(adj_list)`
- `DG = nx.DiGraph(adj_list)`

Python: Create a network with adj matrix

- ```
#adjacency matrix
adj_matrix = np.array([
 [0, 1, 1, 0, 0], # Alice
 [1, 0, 0, 1, 0], # Bob
 [1, 0, 0, 0, 1], # Charlie
 [0, 1, 0, 0, 1], # David
 [0, 0, 1, 1, 0] # Eve
])

Create graph from adjacency matrix
G = nx.from_numpy_array(adj_matrix)

Relabel nodes to match names
node_names = ["Alice", "Bob", "Charlie", "David", "Eve"]
G = nx.relabel_nodes(G, dict(enumerate(node_names)))
```

# Python: Representation

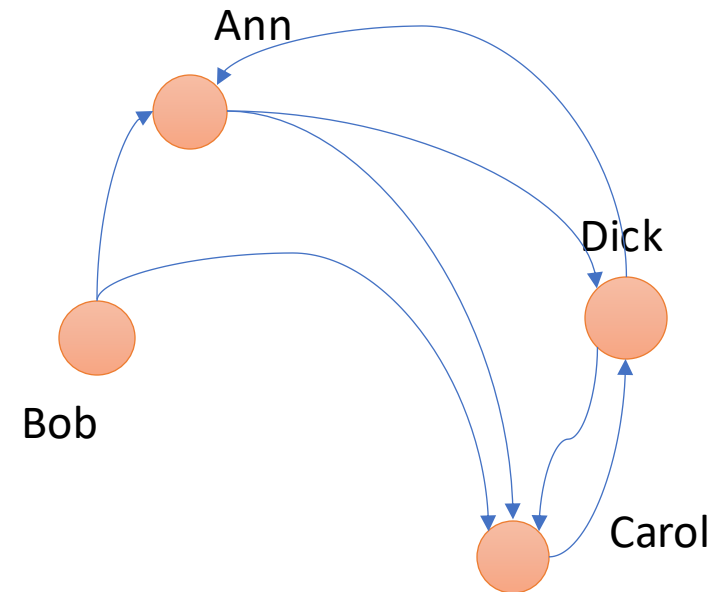
- ```
# Convert to adjacency matrix
adj_matrix = nx.to_numpy_array(G, dtype=int)
print(adj_matrix)

# Convert to edge list
edge_list = list(G.edges())
print(edge_list)
```
- ```
Draw the network
plt.figure(figsize=(6, 6))
nx.draw(G, with_labels=True, node_color="lightblue", edge_color="gray", node_size=700, font_size=12)
plt.title("Simple Social Network")
plt.show()
```
-



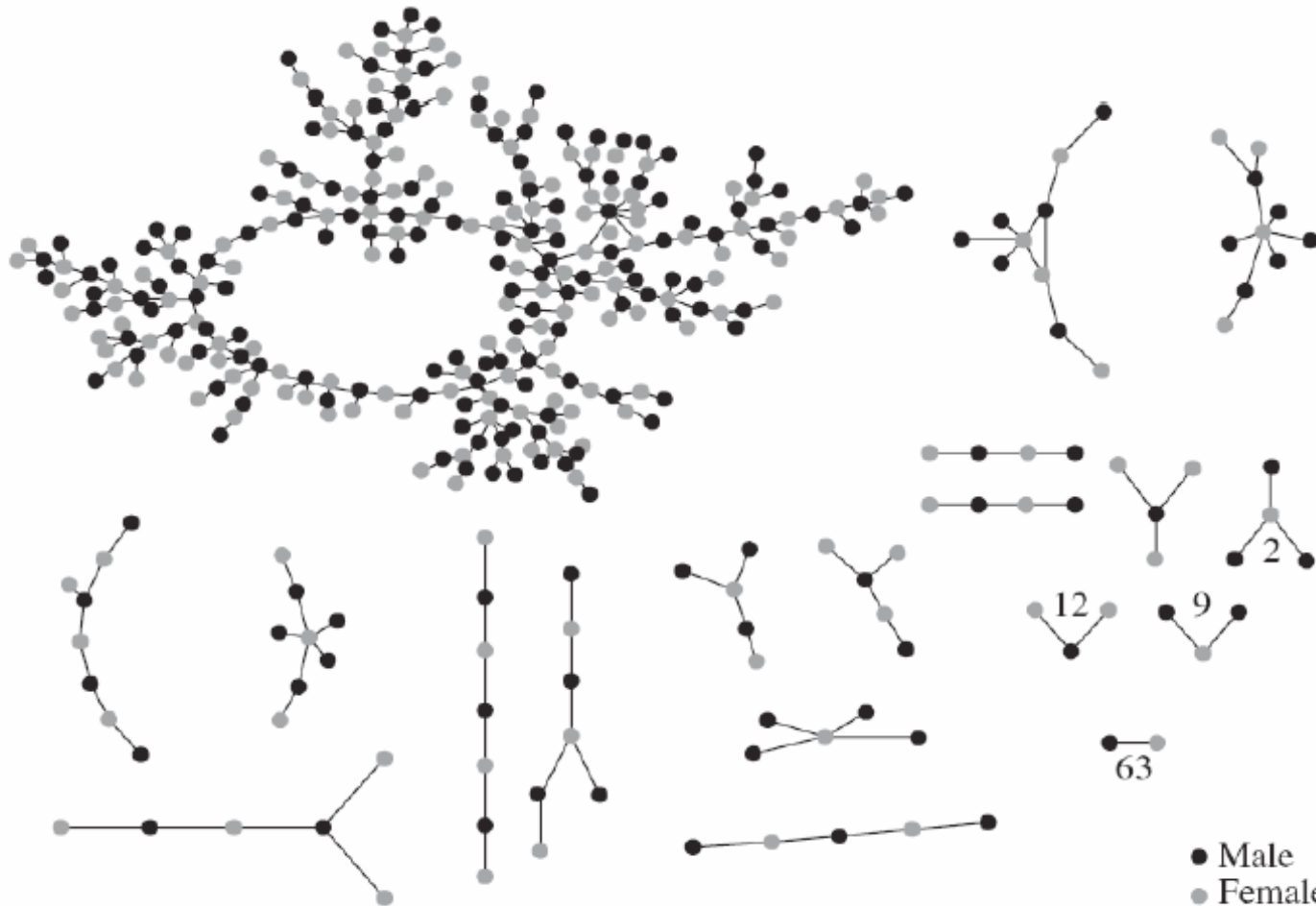
## MGP4b cont'

- Convert your group's social network into an adjacency list
- Convert the adjacency list into an adjacency matrix and edge list
- If your group's social network has no or very few edges, using the directed network example here



# Node types

Bearman, Moody, and Stovel's  
04 High School Romance



- Nodes can have characteristics, called "attributes"
  - Categorical traits, "male", "female"
  - Continuous attributes, age
  - If only one attribute, can be denoted by color

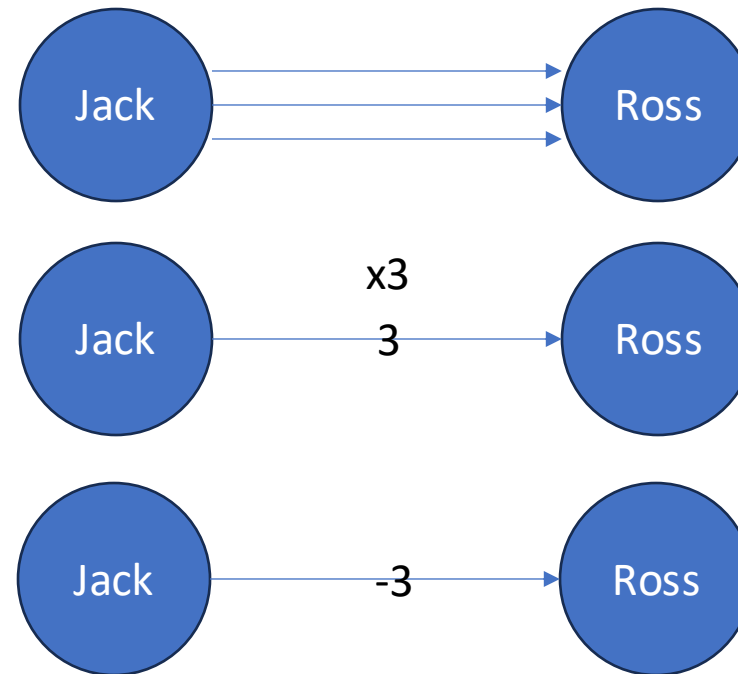
# Python: add node attributes

```
Add nodes without attributes
G.add_nodes_from(["Alice", "Bob", "Charlie",
 "David", "Eve"])
```

```
Add nodes with attributes
G.add_nodes_from([
 ("Alice", {"age": 25, "gender": "F"}),
 ("Bob", {"age": 30, "gender": "M"}),
 ("Charlie", {"age": 22, "gender": "M"}),
 ("David", {"age": 21, "gender": "M"}),
 ("Eve", {"age": 18, "gender": "F"})
])
```

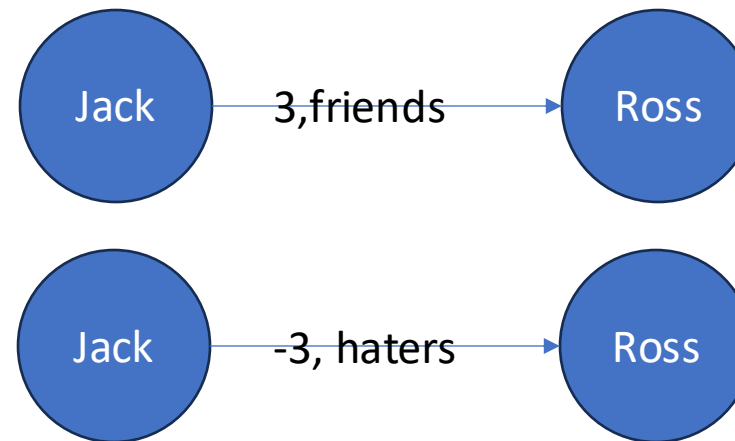
# Weighted social networks

- You want to create a networks of 'likes', not just following
- What would be the nodes?
- What would be the edges?
- What is Jack likes Ross's posts for 3 times?
- What if Jack dislikes Ross's posts for 3 times?



# Edge attributes

- If edge can be measured, can you include other characteristics of edges?
- For example, the types of relationship:
  - Family
  - Friends
  - Colleague



# Python: add edge attributes

- # Add edges (connections between nodes)

```
G.add_edges_from([
 ("Alice", "Bob"),
 ("Alice", "Charlie"),
 ("Bob", "David"),
 ("Charlie", "Eve"),
 ("David", "Eve")
])
```

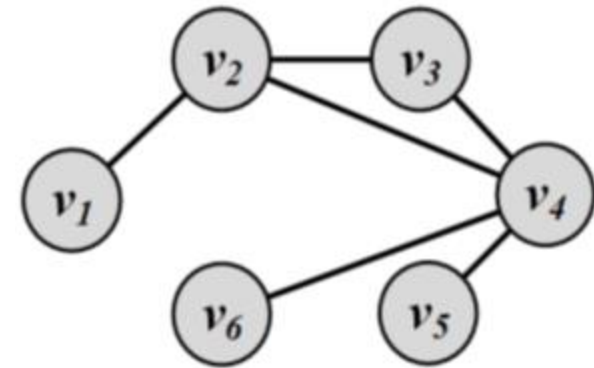
- # Add edges (connections between nodes)

```
G.add_edges_from([
 ("Alice", "Bob", {"relationship": "friend", "weight": 4}),
 ("Alice", "Charlie", {"relationship": "colleague", "weight": -1}),
 ("Bob", "David", {"relationship": "colleague", "weight": 2}),
 ("Charlie", "Eve", {"relationship": "family", "weight": 5}),
 ("David", "Eve", {"relationship": "friend", "weight": -4})
])
```

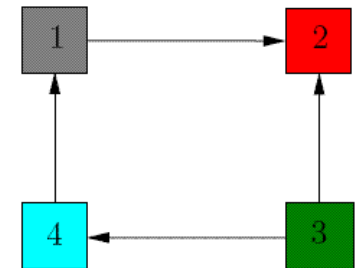
# Basic metrics

- No. of nodes:  
 $|\text{Nodes}| = \text{width/height of adj matrix}$   
 $= \text{dim}(\text{adj matrix})[0] \text{ \#or } [1]$
- No. of (undirected) edges:  
 $|\text{Edges}| = \text{sum}(\text{adj matrix})/2$
- No. of directed edges (arrows):  
 $|\text{Edges}| = \text{sum}(\text{adj matrix})$

|       | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|-------|-------|-------|-------|-------|-------|-------|
| $v_1$ | 0     | 1     | 0     | 0     | 0     | 0     |
| $v_2$ | 1     | 0     | 1     | 1     | 0     | 0     |
| $v_3$ | 0     | 1     | 0     | 1     | 0     | 0     |
| $v_4$ | 0     | 1     | 1     | 0     | 1     | 1     |
| $v_5$ | 0     | 0     | 0     | 1     | 0     | 0     |
| $v_6$ | 0     | 0     | 0     | 1     | 0     | 0     |



|   | 0 | 1 | 0 | 0 |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 0 |



# Python

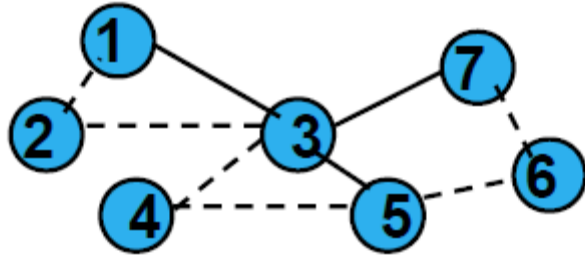
- ```
# Basic Network Statistics  
num_nodes = G.number_of_nodes()  
num_edges = G.number_of_edges()
```

```
print(num_nodes)  
print(num_edges)
```

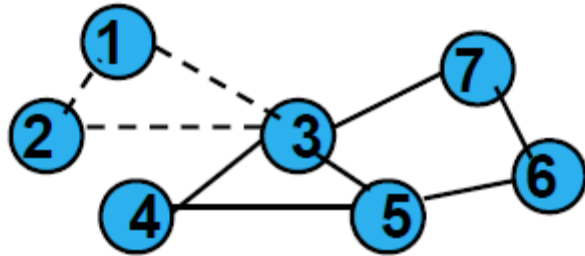

Additional definitions

- **Walk from i_1 to i_k :** a sequence of nodes $(i_1, i_2, \dots, i_k)$ and sequence of edges $(i_1i_2, i_2i_3, \dots, i_{k-1}i_k)$ such that $i_{k-1}i_k$ is an edge (can be directed) in g (g is the graph) for each k
- **Path:** a walk $(i_1, i_2, \dots, i_k)$ with each node i_k distinct
- **Cycle:** a **walk** where $i_1 = i_k$
- **Simple cycle:** a **path** where $i_1 = i_k$
- **Geodesic:** a shortest **path** between two nodes
- **Geodesic distance:** the number of edges on the geodesic

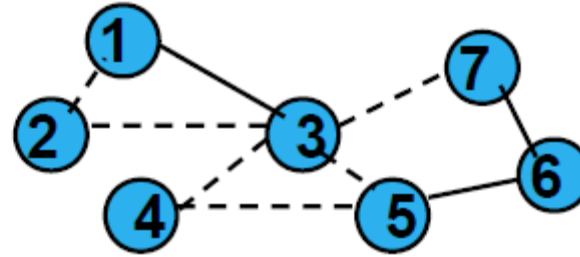
Distinctions



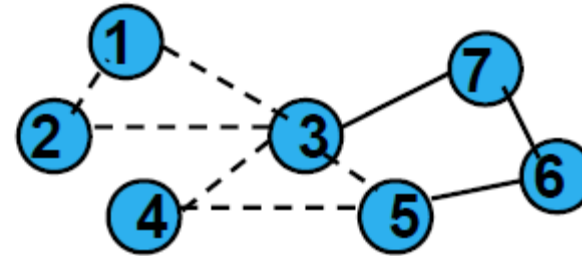
Path (and a walk) from 1 to 7:
1, 2, 3, 4, 5, 6, 7



Simple Cycle (and a walk)
from 1 to 1: 1, 2, 3, 1



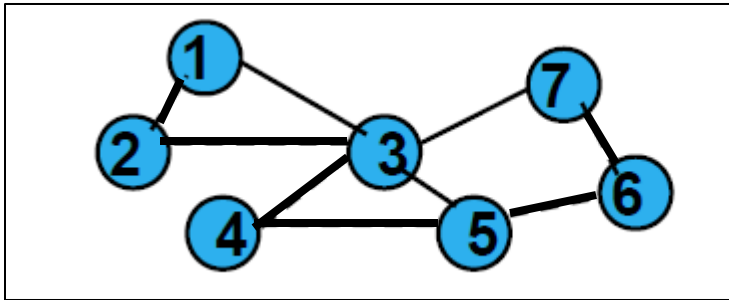
Walk from 1 to 7 that is not a path:
1, 2, 3, 4, 5, 3, 7



Cycle (and a walk) from 1 to 1:
1, 2, 3, 4, 5, 3, 1

Simple cycle: a cycle with no repetition of vertices and edges.
The definition of a simple cycle is NOT required in exams.

Practice



How many edges are there?

9

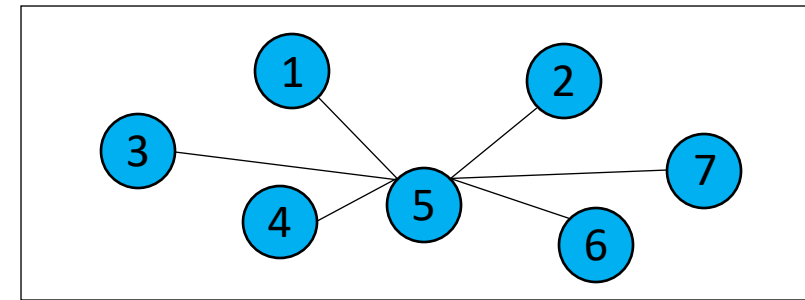
What is the **Geodesic** from 1 to 6?

1-3-5-6

1-3-7-6

What is the **Geodesic distance** from 1 to 6?

3



How many edges are there?

6

What is the **Geodesic** from 1 to 6?

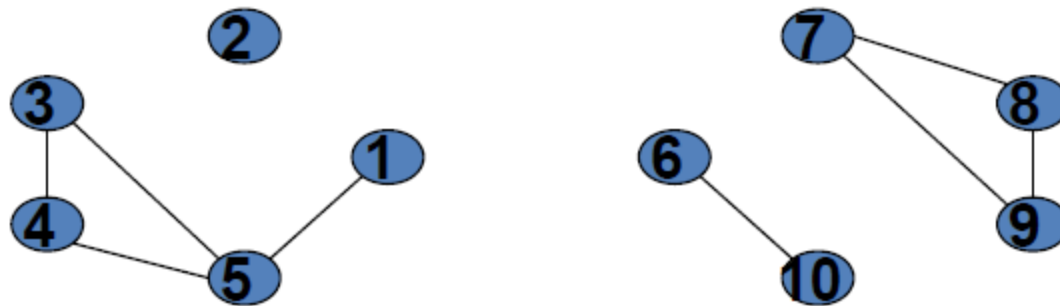
1-5-6

What is the **Geodesic distance** from 1 to 6?

2

Components

- A graph is **connected** if there is a **path** between every two nodes
- This means that no node is isolated, and every node can be reached from any other node.
- Component: maximal **connected** subgraph



A network with four components

Directed network

- A directed graph is called **strongly connected** if starting at any node i we can reach any other different node j by walking on its edges.

